Machine Learning and Data Processing

a.a. 2024-2025

Natural Language Processing (NLP)

Analisi e applicazioni del linguaggio naturale

Prof. Ing. F. Benedetto

francesco.benedetto@uniroma3.it
fbenedetto@ieee.org

Chair of "Machine Learning and Data Processing" – University of Roma Tre
Chair of the "Definitions and Concepts for Dynamic Spectrum Access" - IEEE 1900.1

©SP4TE Lab 2017

Agenda

1. **Cos'è il NLP?**
   - *Storia del NLP*
   - *Principali tecnologie e approcci*
2. **Preprocessing del linguaggio**
   - *Modelli di linguaggio*
   - *Tokenizzazione*
   - *Stemming e Lemmatizzazione*
   - *Word Embedding*
   - *Trasformatori (Transformers)*
3. **Reti neurali nel NLP**
   - *NLP e Machine Learning*
   - *Sentiment Analysis*
4. **Sfide attuali del NLP**
   - *Etica e bias nel NLP*
5. **Conclusione**

NLP: cos'è

Natural Language Processing (NLP) è una disciplina dell'intelligenza artificiale (AI) che si occupa di elaborare, comprendere e generare il linguaggio umano utilizzando i computer.

Obiettivo principale: colmare il divario tra linguaggio umano (non strutturato) e linguaggio macchina (strutturato).

Domini di Applicazione

Comprensione del linguaggio naturale (Natural Language Understanding - NLU): analisi di significati, sentimenti e intenzioni da un testo.

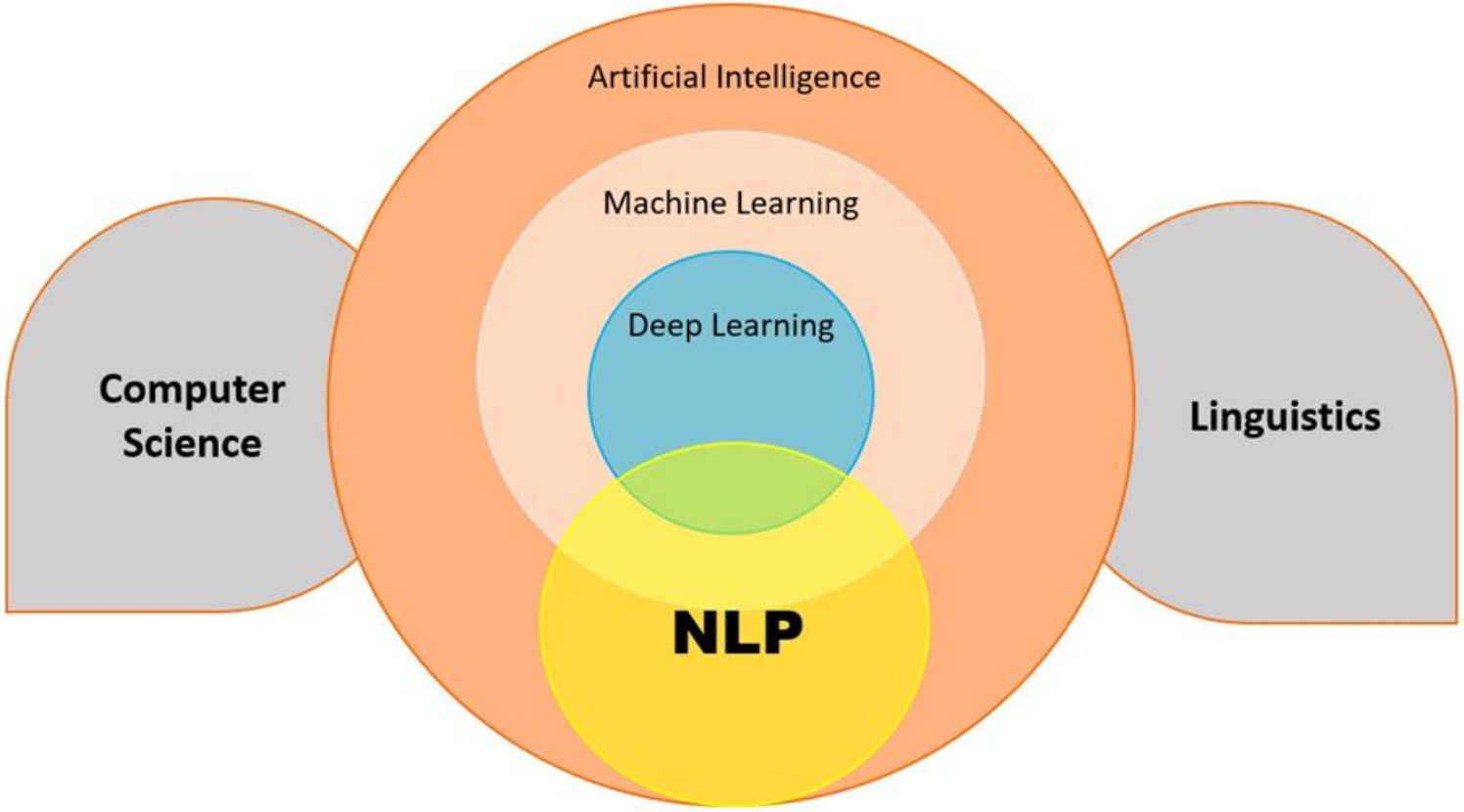
Generazione del linguaggio naturale (Natural Language Generation - NLG): produzione di testo simile a quello scritto da un essere umano.

Perché è Importante?

Il linguaggio umano è la forma più naturale di comunicazione, ma è complesso, ambiguo e ricco di sfumature.

Il NLP consente di:

- ✓ Automatizzare processi basati sul linguaggio.
- ✓ Estrarre informazioni utili da grandi quantità di dati testuali.
- ✓ Migliorare l'interazione uomo-macchina.



NLP: cos'è

Componenti Principali

- ✓ **Analisi sintattica**: studio della struttura grammaticale delle frasi.
- ✓ **Analisi semantica**: comprensione del significato delle parole e delle frasi.
- ✓ **Analisi pragmatica**: interpretazione del linguaggio in base al contesto.

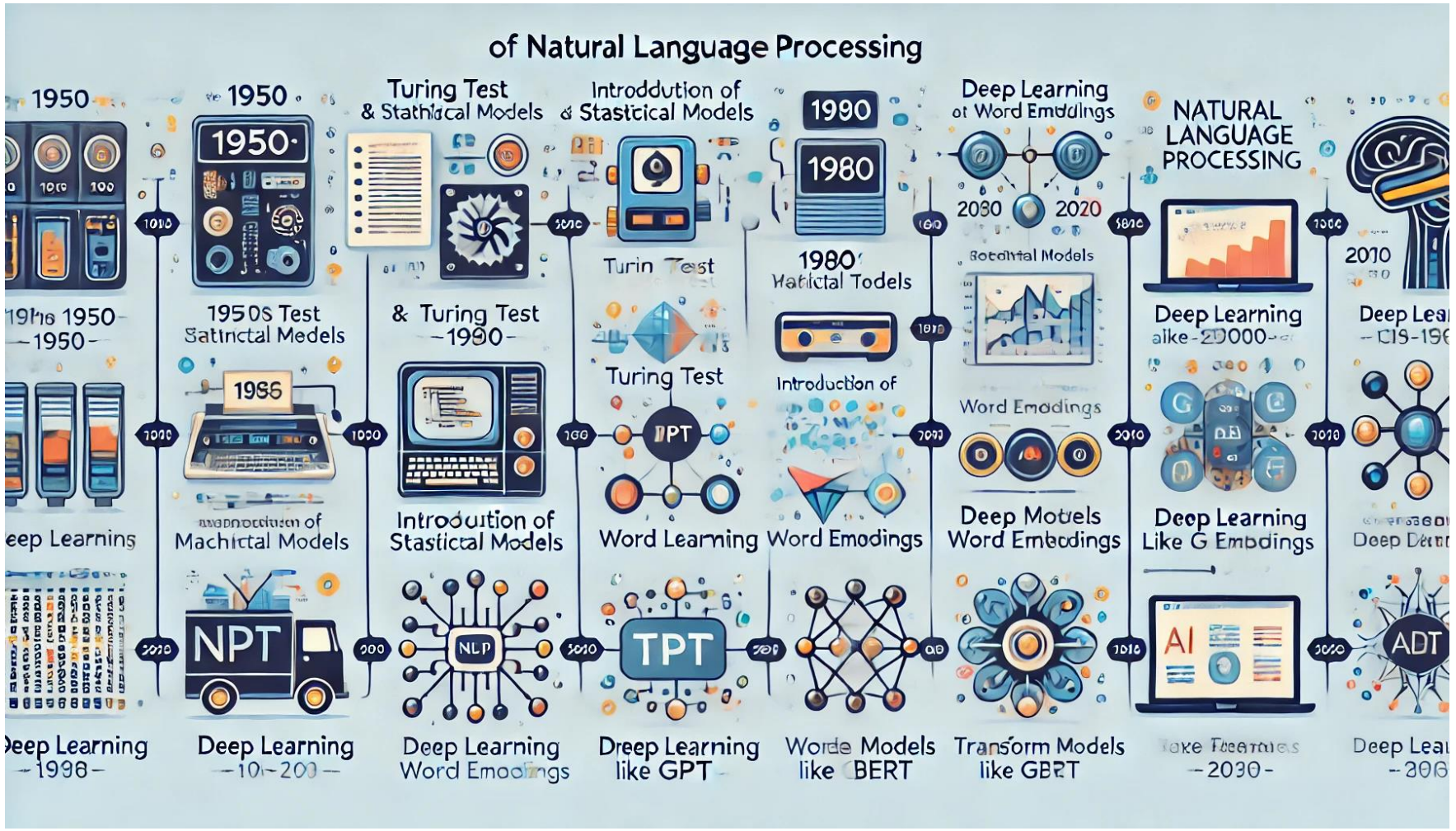
Esempi Pratici

- ✓ Motori di ricerca (Google, Bing).
- ✓ Traduttori automatici (DeepL, Google Translate).
- ✓ Assistenti virtuali (Siri, Alexa, Google Assistant).
- ✓ Analisi di sentimenti nei social media o nelle recensioni

NLP: un po' di storia

Origini e Sviluppo

- **1950s:**
  - Alan Turing introduce il concetto di "macchine pensanti" con il famoso **Test di Turing**.
  - Primo tentativo di traduzione automatica con sistemi basati su regole (esempio: Progetto Georgetown-IBM).
- **1960s-1970s:**
  - L'uso di grammatiche formali (come quelle di Chomsky) domina l'elaborazione linguistica.
  - I sistemi esperti vengono sviluppati per elaborare comandi specifici.
- **1980s:**
  - Introduzione di modelli statistici, come i modelli Markoviani (Hidden Markov Models - HMM).
  - Focus su linguistica computazionale e primi passi verso l'apprendimento automatico.



NLP: un po' di storia

Evoluzione Moderna

•1990s-2000s:

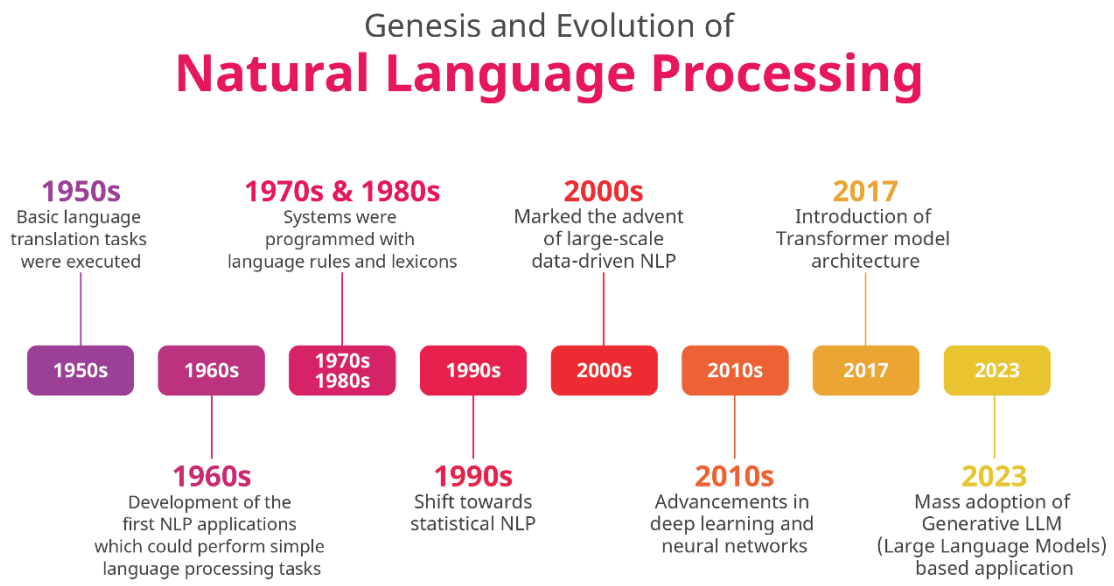
- Sviluppo di algoritmi statistici più avanzati e l'uso di grandi corpora testuali.
- Inizio dell'uso commerciale con motori di ricerca e correttori ortografici.

•2010s:

- Rivoluzione del deep learning nel NLP: reti neurali profonde migliorano la comprensione del contesto e la generazione di testo.
- Introduzione di tecniche di **Word Embedding** (es. Word2Vec, GloVe).

•2020s:

- Avvento dei modelli **Transformers**, come BERT, GPT e altri, che ridefiniscono lo stato dell'arte nel NLP.
- Applicazioni sempre più sofisticate: traduzione automatica di alta qualità, assistenti vocali, e modelli multimodali.

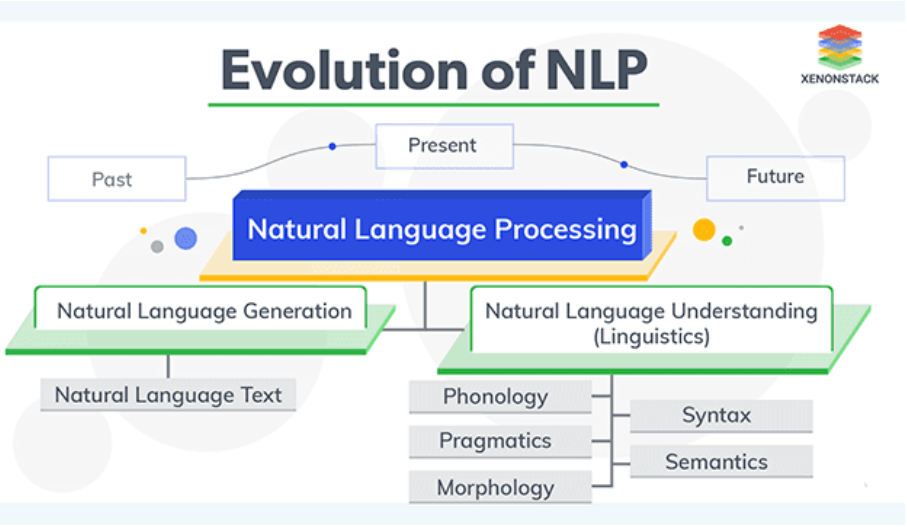


Sfide Passate

•Mancanza di potenza computazionale nei primi decenni.

•Difficoltà nel gestire la complessità semantica e pragmatica del linguaggio umano.

•Disponibilità limitata di corpora di dati per addestrare i modelli.



NLP: tecnologie e approcci

1. Approcci Basati su Regole
I primi sistemi NLP utilizzavano regole grammaticali scritte a mano per analizzare e generare linguaggio.

- ✓ Strutture grammaticali rigide.
- ✓ Dipendenza dall'esperienza linguistica dagli sviluppatori.

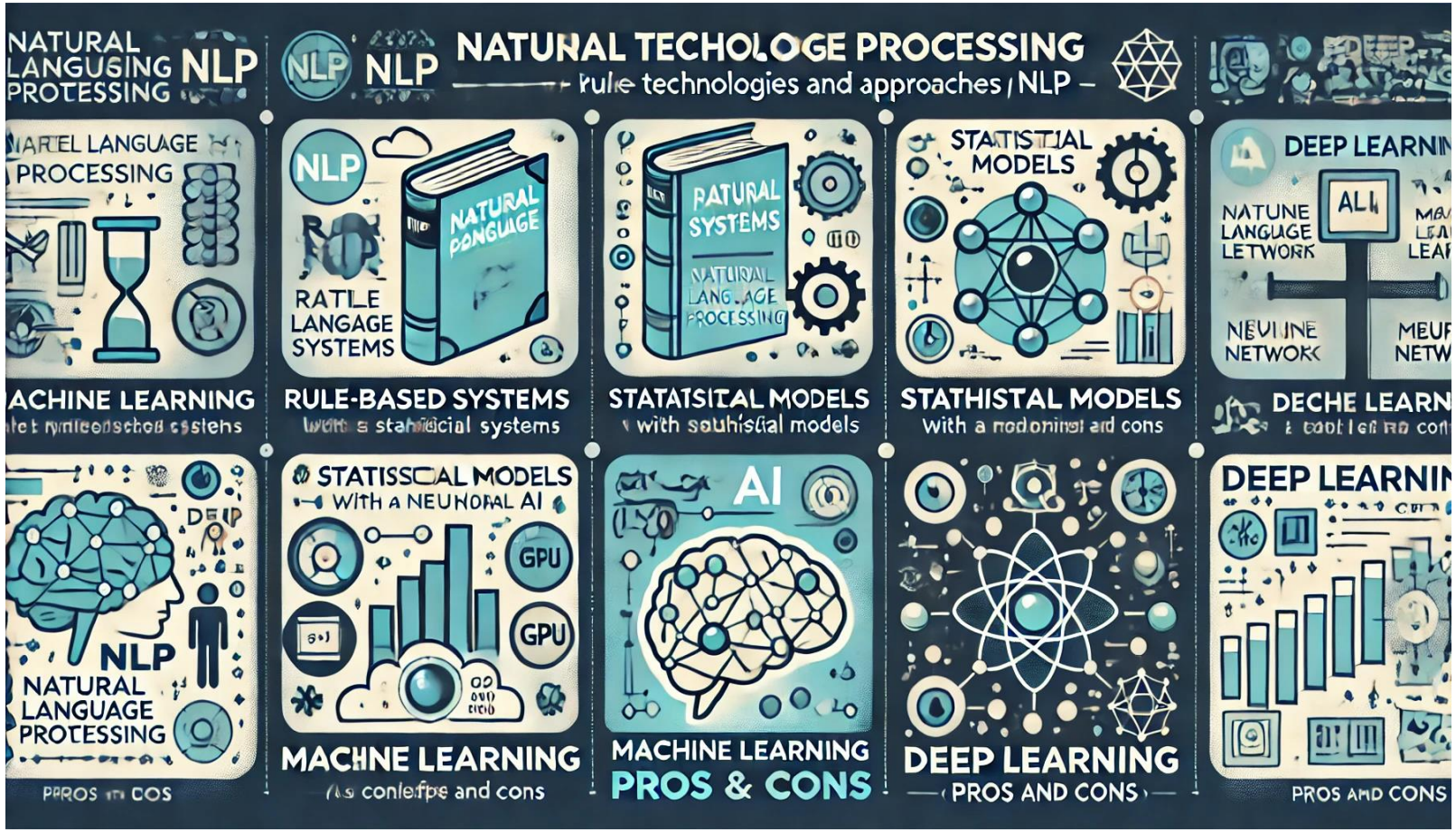
- ✓ Semplici da implementare per lingue con regole ben definite.
- ✓ Poco scalabili.
- ✓ Non adatti a gestire ambiguità o variazioni linguistiche.
- ✓ Elevata manutenzione.

2. Modelli Statistici
Utilizzano metodi probabilistici per analizzare il linguaggio basandosi su grandi corpora di dati testuali.

- ✓ Modelli di Markov.
- ✓ Modelli N-gram per prevedere la probabilità di sequenze di parole.

- ✓ Miglior gestione delle variazioni linguistiche.
- ✓ Non richiedono regole scritte a mano.

- ✓ Richiedono grandi quantità di dati.
- ✓ Limitati nella comprensione del contesto semantico.



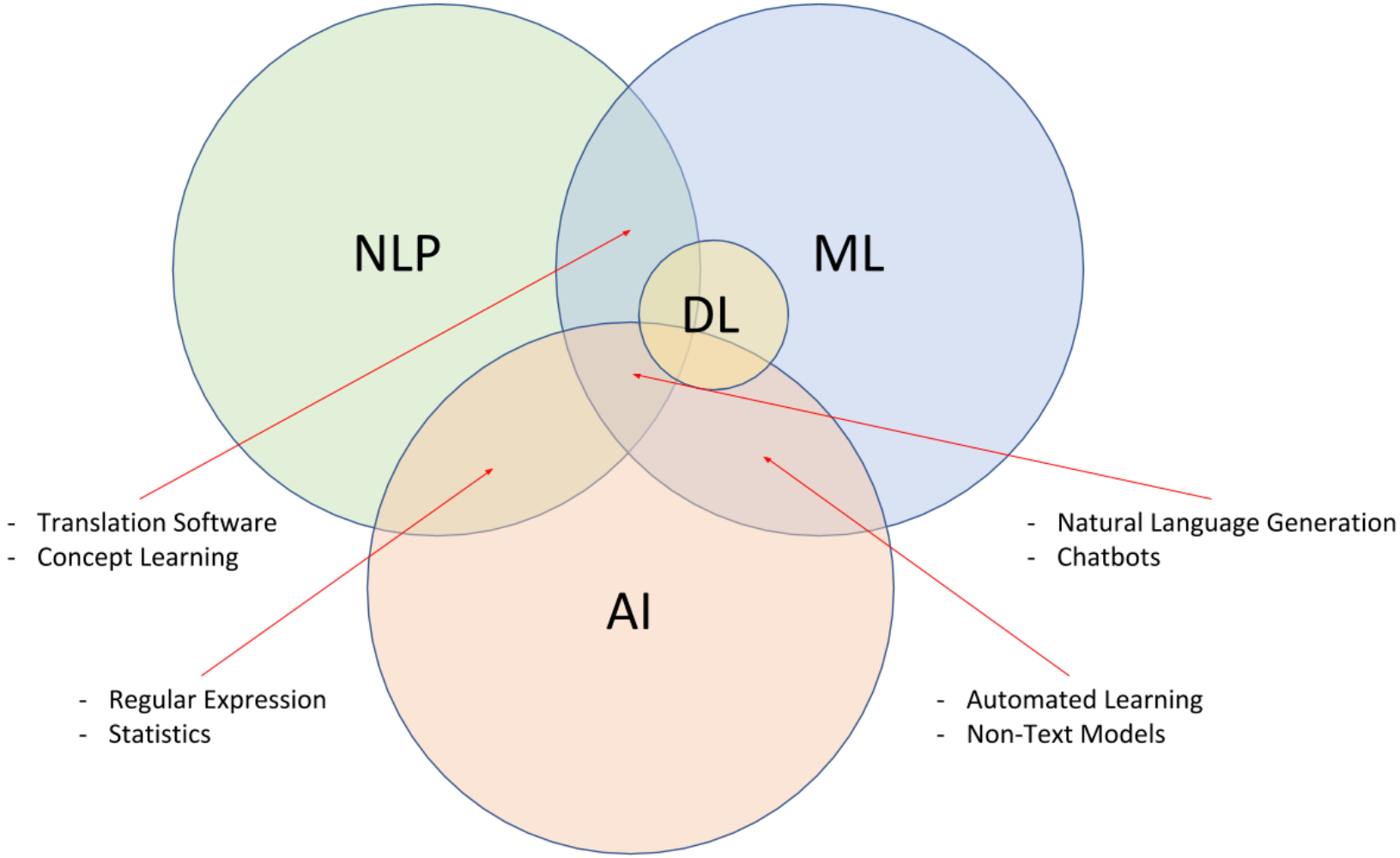
NLP e Machine Learning

Approcci Basati su Machine Learning
I modelli apprendono le caratteristiche del linguaggio da dati etichettati.

- ✓ Alberi decisionali.
- ✓ Support Vector Machines (SVM).
- ✓ Reti neurali.

- ✓ Più flessibili rispetto ai metodi basati su regole.
- ✓ Adatti a vari tipi di problemi NLP.
- ✓ Richiedono dati annotati di alta qualità.
- ✓ Complessità computazionale.

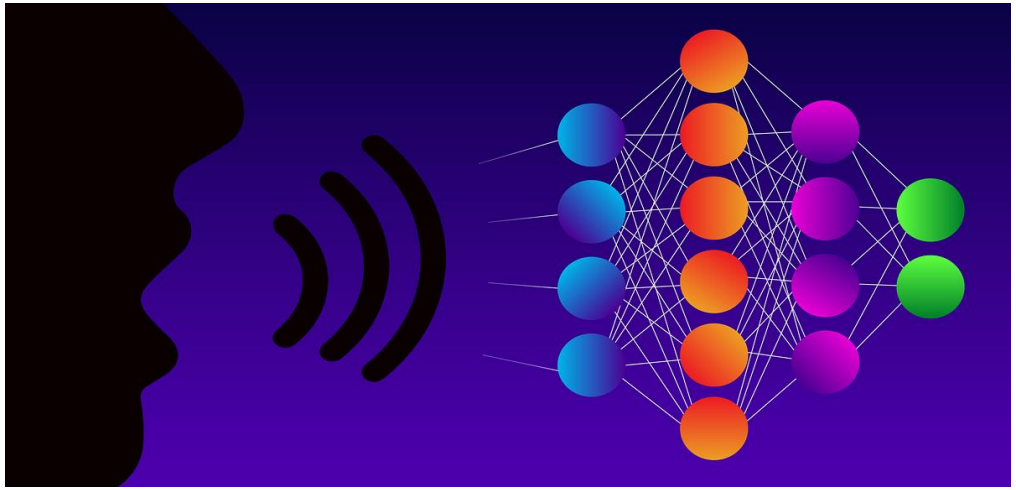
Esempi di applicazione:
- ✓ Classificazione del testo (es. spam/non-spam).
- ✓ Riconoscimento delle entità nominate (Named Entity Recognition - NER).



NLP e Deep Learning

Modelli basati su reti neurali profonde, in grado di apprendere rappresentazioni complesse e contestuali del linguaggio.

- **Tecniche principali**:
  - Reti Neurali Ricorrenti (RNN).
  - Long Short-Term Memory (LSTM).
  - Transformers (es. GPT, BERT).
- **Vantaggi**:
  - Comprensione avanzata del contesto e delle relazioni semantiche.
  - Risultati all'avanguardia in molte applicazioni NLP.
- **Svantaggi**:
  - Elevato costo computazionale.
  - Necessità di grandi dataset per l'addestramento.



Schema Riassuntivo

Approccio	Vantaggi	Svantaggi
Basato su regole	Semplice, intuitivo	Poco scalabile, rigido
Statistico	Gestisce variazioni linguistiche	Non comprende il contesto
Machine Learning	Flessibile, adattabile	Richiede dati annotati
Deep Learning	Elevata precisione, contestuale	Costo elevato, complesso

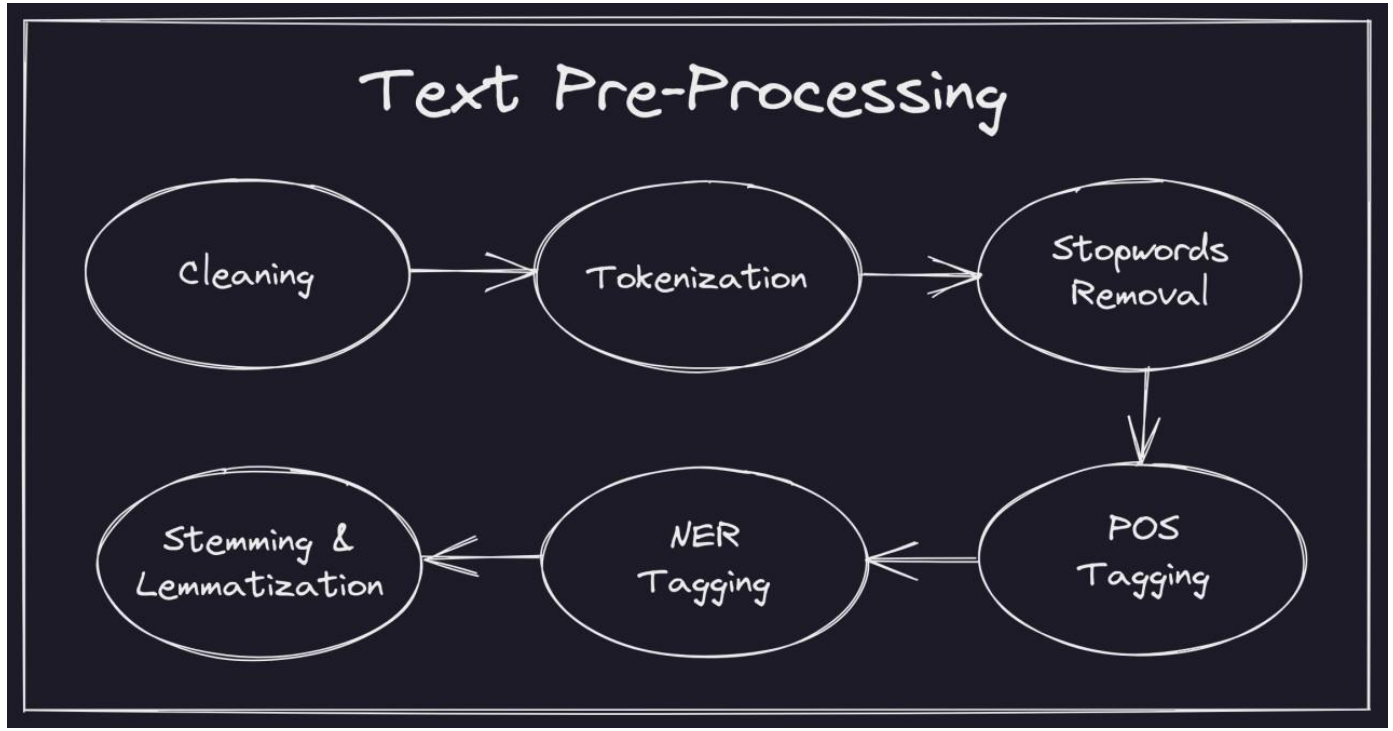
NLP: Pre-processing

Il preprocessing è il primo passo nell'elaborazione del linguaggio naturale, in cui il testo grezzo viene trasformato in un formato che i modelli possono analizzare e utilizzare.

Obiettivo: pulire, normalizzare e strutturare i dati per migliorare le prestazioni degli algoritmi NLP.

Principali Tecniche di Preprocessing:

- ✓ Pulizia del Testo: Rimuovere caratteri speciali, numeri, eccessi di spazi.
- ✓ Uniformare maiuscole/minuscole (es. trasformare tutto il testo in minuscolo), per garantire una base omogenea per l'elaborazione.
- ✓ Rimozione di Stop Words: parole comuni che non aggiungono significato sostanziale (es. "il", "e", "di"). Rimuoverle aiuta a ridurre il rumore nei dati, migliorando la rilevanza delle analisi.



NLP: Pre-processing

- ✓ Tokenizzazione: Divisione del testo in unità più piccole chiamate token (parole, frasi o caratteri).

  Tipi di tokenizzazione:
  Tokenizzazione per parola: il testo viene diviso in parole.
  Tokenizzazione per frase: il testo viene diviso in frasi.
  Subword tokenization: utile per lingue con morfologia complessa.

- ✓ Normalizzazione: Trasformare parole con grafie diverse ma significati uguali in una forma standard (es. "camminavo" → "camminare"). Inclusa anche la conversione di sinonimi o abbreviazioni a una forma standard.

- ✓ Stemming e Lemmatizzazione

  Stemming: riduzione di una parola alla sua radice (es. "camminavo", "cammina" → "cammin").
  Lemmatizzazione: riduzione di una parola alla sua forma base o lemma (es. "camminavo" → "camminare").
  Differenza chiave: la lemmatizzazione considera il contesto grammaticale.

- ✓ Rimozione di Punteggiatura

  Esclude simboli come ".", ",", "!" che non aggiungono valore semantico. Utile per alcune applicazioni, ma non per tutte (es. analisi sintattica).

Il Preprocessing migliora l'accuratezza e l'efficienza dei modelli di machine learning e deep learning. Consente di ridurre il rumore nei dati, enfatizzando informazioni rilevanti. È un passaggio fondamentale per applicazioni come sentiment analysis, traduzione automatica e classificazione del testo.

NLP: Tokenizzazione

E’ il processo di suddividere un testo in unità più piccole chiamate token.

Questi token possono essere parole, frasi, o anche caratteri, a seconda del tipo di tokenizzazione utilizzato. È una fase cruciale del preprocessing perché consente ai modelli NLP di lavorare su unità linguistiche più comprensibili e gestibili.

Tokenizzazione per Parola: Il testo viene suddiviso in singole parole.

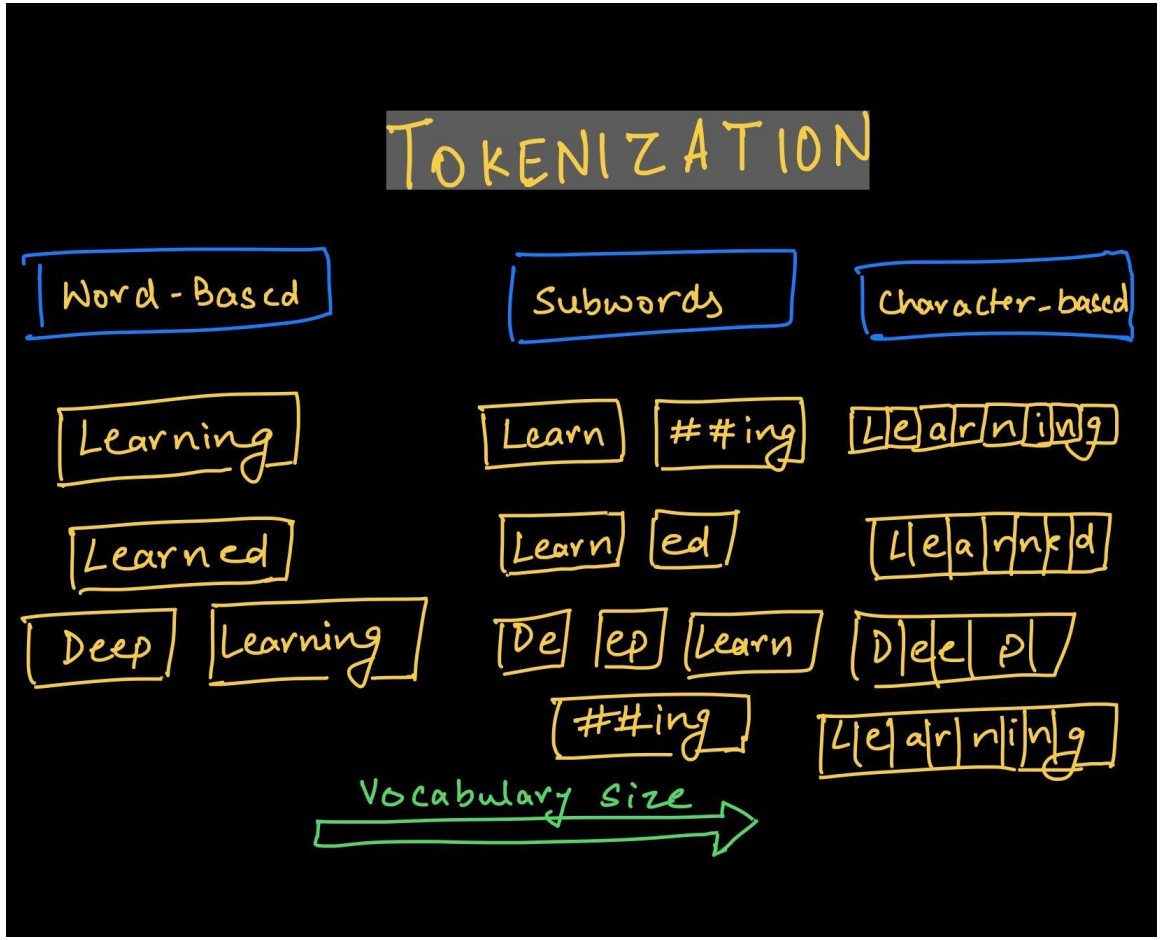
- Esempio: Testo: «Ciao, come stai?» Token: ["Ciao", "come", "stai"] Questa è la forma di tokenizzazione più comune per molte applicazioni NLP come la classificazione del testo.

Tokenizzazione per Frase: Il testo viene suddiviso in frasi.

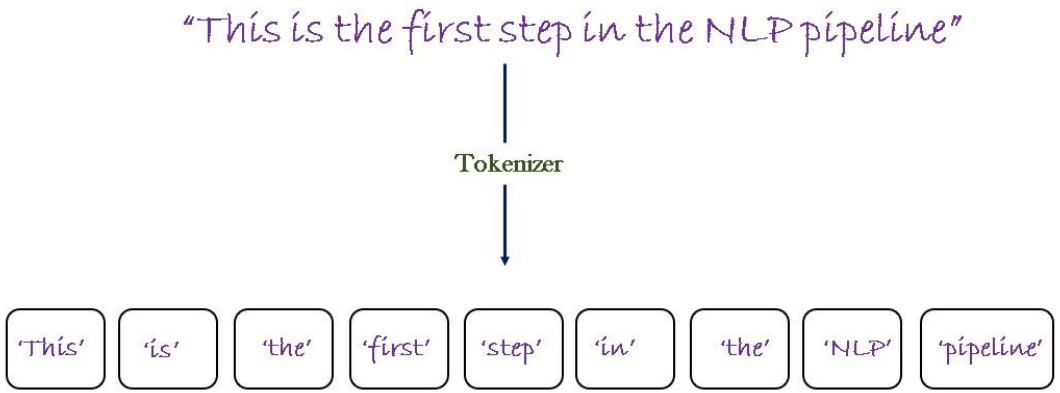
- Esempio:Testo: «Ciao, come stai? Spero tutto bene!» Token: ["Ciao, come stai?", "Spero tutto bene!"] Utile per applicazioni come l'analisi del sentiment a livello di frase.

Tokenizzazione per Subword (Sottoparola): Le parole vengono suddivise in unità più piccole come sottoparole (morfemi).

- Esempio: Parola: "comunicazione« Token: ["comu", "nic", "azione"] Approccio utilizzato in modelli avanzati come BERT e GPT per gestire parole sconosciute o nuove.



NLP: Tokenizzazione



Tokenizzazione e Sfide Linguistiche

Lingue con Spazi: In lingue come l'italiano, la tokenizzazione per parola è semplice, poiché le parole sono separate da spazi.

Lingue senza Spazi: In lingue come il cinese, giapponese o arabo, la tokenizzazione è molto più complessa, poiché non esistono spazi tra le parole.

Ambiguità: La tokenizzazione deve anche gestire casi di ambiguità (es. "non vedo" potrebbe essere un'unica unità o due).

Contrazioni: In inglese, contrazioni come "I'm" o "they're" devono essere gestite correttamente.

Tecniche Avanzate di Tokenizzazione

Byte Pair Encoding (BPE): Un algoritmo che suddivide le parole in segmenti più piccoli, ripetendo iterativamente la divisione dei byte più comuni. Utile per lingue con parole complesse.

WordPiece (utilizzato in BERT): Una variante di BPE, che divide le parole in sottoparole, migliorando la capacità del modello di gestire parole non presenti nel vocabolario.

SentencePiece: Tecnica che non richiede pre-segmentazione e può essere usata su dati non segmentati, utile per lingue senza spazi.

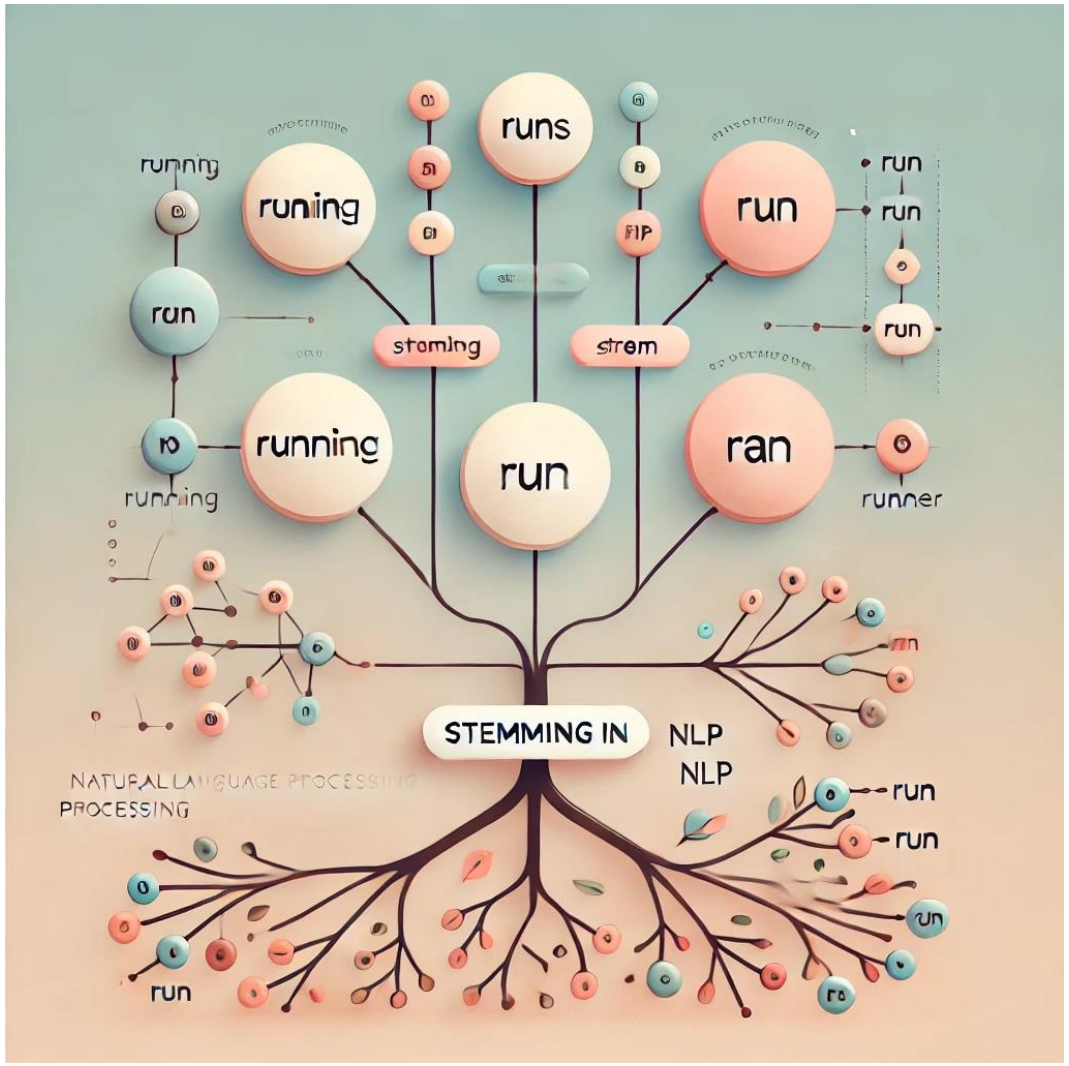
NLP: Stemming

Stemming: Lo stemming è un processo di normalizzazione che riduce le parole alla loro radice (stem).

- Non tiene conto della grammatica o del significato, ma è utile per ridurre la dimensionalità del testo.

- Limitazioni del processo di Stemming
  - Perdita di significato: Lo stemming non mantiene il significato esatto delle parole.
  - Ambiguità: Parole diverse possono essere ridotte alla stessa radice erroneamente. Esempio: "studying" e "study" → "studi«

- Applicazioni
  - Sistemi di recupero delle informazioni (Information Retrieval).
  - Motori di ricerca per migliorare la corrispondenza delle query.
  - Preprocessing per modelli di Machine Learning.



NLP: Stemming – codice python

```
from nltk.stem import PorterStemmer

# Inizializza lo stemmer
stemmer = PorterStemmer()

# Lista di parole di esempio
words = ["camminare", "camminato", "camminando", "cammina", "camminano"]

# Esegui stemming
stemmed_words = [stemmer.stem(word) for word in words]

# Stampa i risultati
print("Parole originali:", words)
print("Stemming:", stemmed_words)
```

OUTPUT

```
Parole originali: ['camminare', 'camminato', 'camminando',
'cammina', 'camminano']
Stemming: ['camminar', 'camminat', 'camminando', 'cammina',
'camminano']
```

NLP: Lemmatizzazione

La **lemmatizzazione** è un processo di normalizzazione delle parole che trasforma una parola alla sua forma base o radice, detta **lemma**. A differenza dello stemming, la lemmatizzazione considera il contesto grammaticale e il significato della parola.

•**Esempio**:

- *Corsi → correre*
- *Mangiando → mangiare*
- *Gatti → gatto*

La lemmatizzazione utilizza dizionari morfologici per identificare il lemma corretto.
Richiede l'identificazione del parte del discorso (Part of Speech, POS), come:
Nomi (NOUN) - Verbi (VERB) - Aggettivi (ADJ)

Caratteristica	Stemming	Lemmatizzazione
Approccio	Rimuove suffissi basandosi su regole	Analizza la parola nel contesto
Precisione	Meno precisa	Più precisa
Output	Radice "grezza"	Lemma corretto
Esempio (English)	*Running → runn*	*Running → run*

Vantaggi della Lemmatizzazione

Maggiore precisione rispetto allo stemming.
Riduzione della variabilità delle parole.

Limiti della Lemmatizzazione

Richiede più risorse computazionali rispetto allo stemming.
Dipende dalla lingua e dal contesto grammaticale.
Necessita di modelli linguistici pre-addestrati (es. spaCy, WordNet).

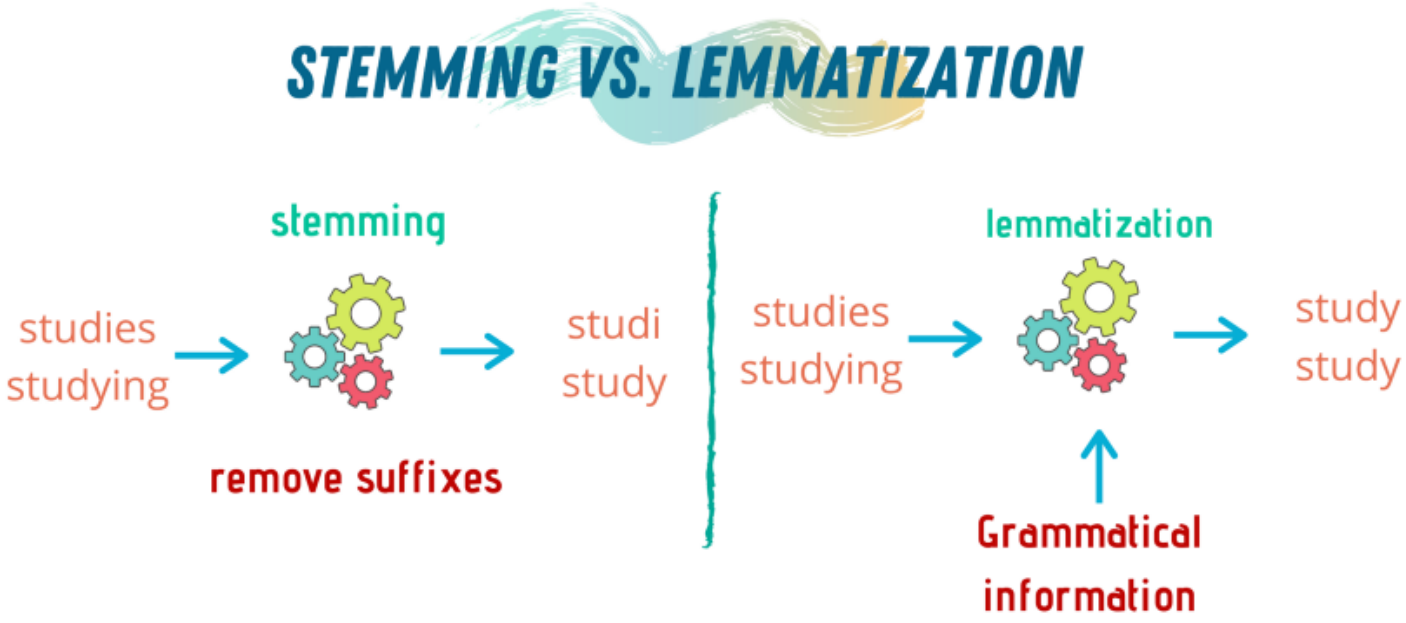
NLP: Lemmatizzazione – codice python

```
import spacy

# Carica il modello della lingua italiana
nlp = spacy.load("it_core_news_sm")

# Testo di esempio
text = "Corsi, mangiando, giocano, gatti"

# Applica la lemmatizzazione
doc = nlp(text)
print("Parole originali e lemmatizzate:")
for token in doc:
    print(f"{token.text} → {token.lemma_}")
```



OUTPUT

Corsi → correre
mangiando → mangiare
giocano → giocare
gatti → gatto

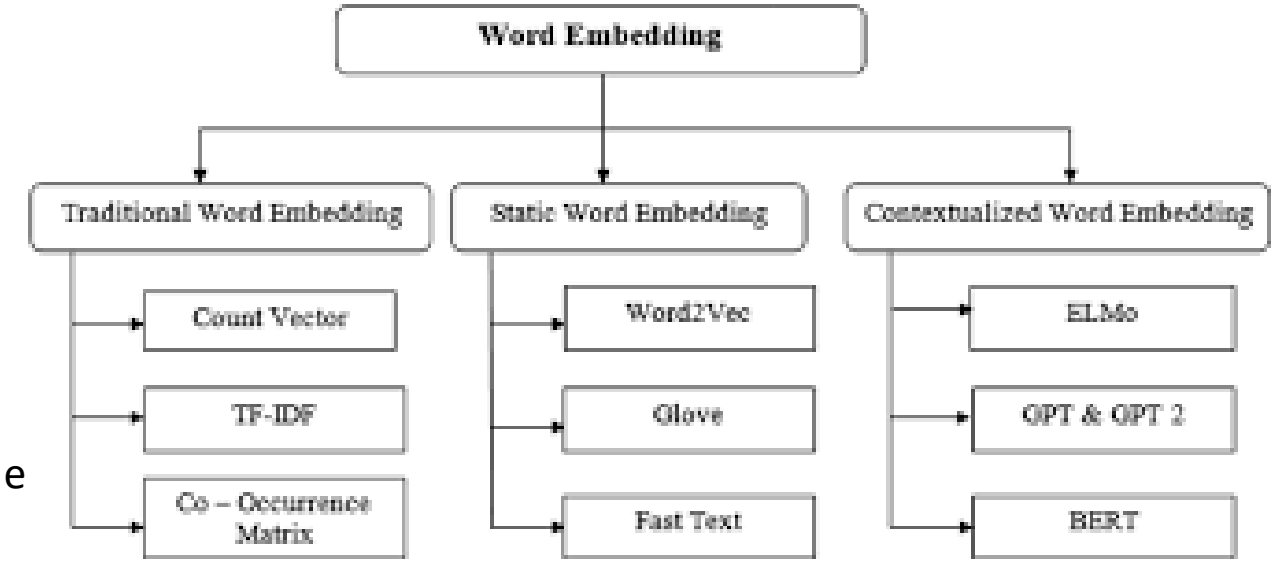
NLP: word embedding

Il Word Embedding è una tecnica di rappresentazione delle parole in formato vettoriale utilizzando modelli matematici e statistici.

Ogni parola viene trasformata in un vettore di numeri reali di dimensione fissa, dove parole con significati simili sono rappresentate da vettori vicini nello spazio.

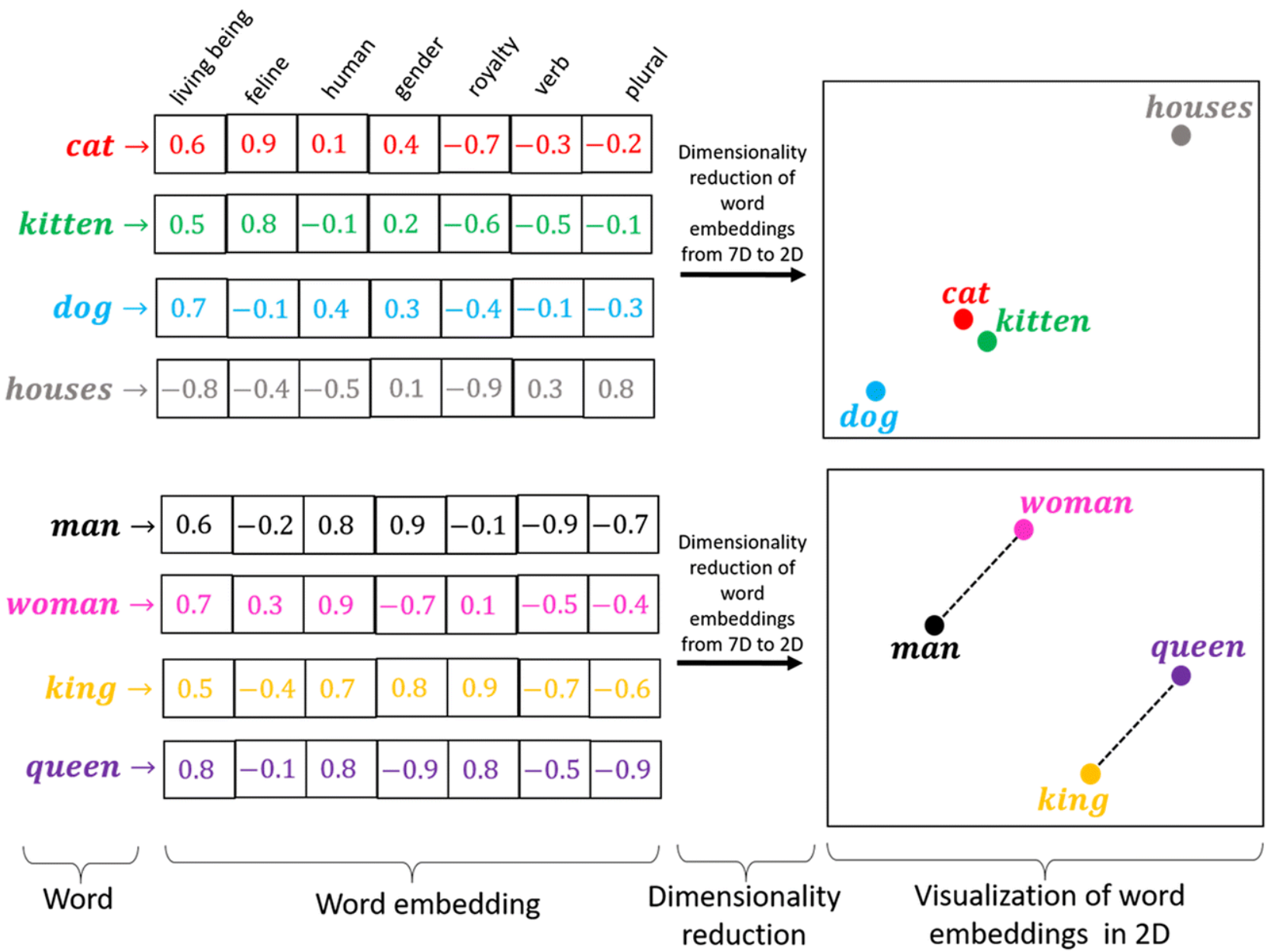
Parole simili come "gatto" e "felino" avranno rappresentazioni vettoriali vicine.

- ✓ Cattura le relazioni semantiche tra parole.
- ✓ Riduce la dimensionalità del testo.
- ✓ Rende le parole interpretabili dai modelli di Machine Learning e Deep Learning.



Tecnica	Descrizione
Word2Vec	- Modello predittivo. Addestra embedding attraverso contesti (CBOW o Skip-gram).
GloVe	- Modello basato su conteggio delle co-occorrenze delle parole.
FastText	- Estende Word2Vec includendo sotto-parole, utile per parole rare o non viste.
Embedding pretrained	- Utilizza embedding pre-addestrati su enormi dataset (es. GloVe, spaCy, BERT).

NLP: word embedding



Limiti del Word Embedding

Out-of-Vocabulary (OOV): Parole non viste durante l'addestramento non hanno vettori.

Perdita di contesto: Parole ambigue (es. banco come "fiume" o "finanza") possono avere un solo embedding.

Modelli statici: Word2Vec e GloVe non catturano il contesto dinamico come nei modelli più recenti (es. BERT).

NLP: word embedding – codice python

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize

# Testo di esempio
sentences = [
    "Il gatto salta sul tavolo",
    "Il cane dorme sulla poltrona",
    "Gatti e cani sono animali domestici"
]

# Tokenizza il testo
tokenized_sentences = [word_tokenize(sentence.lower()) for sentence in
sentences]

# Addestra il modello Word2Vec
model = Word2Vec(sentences=tokenized_sentences, vector_size=100,
window=5, min_count=1)

# Ottieni il vettore di una parola
vector_gatto = model.wv['gatto']
print("Vettore per la parola 'gatto':", vector_gatto)

# Trova parole simili
similar_words = model.wv.most_similar('gatto', topn=3)
print("Parole simili a 'gatto':", similar_words)
```

OUTPUT

Parole simili a 'gatto': [('cane', 0.85), ('animali', 0.75), ('domestici', 0.70)]

NLP: Transformers

Il Transformer è un'architettura di rete neurale introdotta nel paper "Attention Is All You Need" (Vaswani et al., 2017).

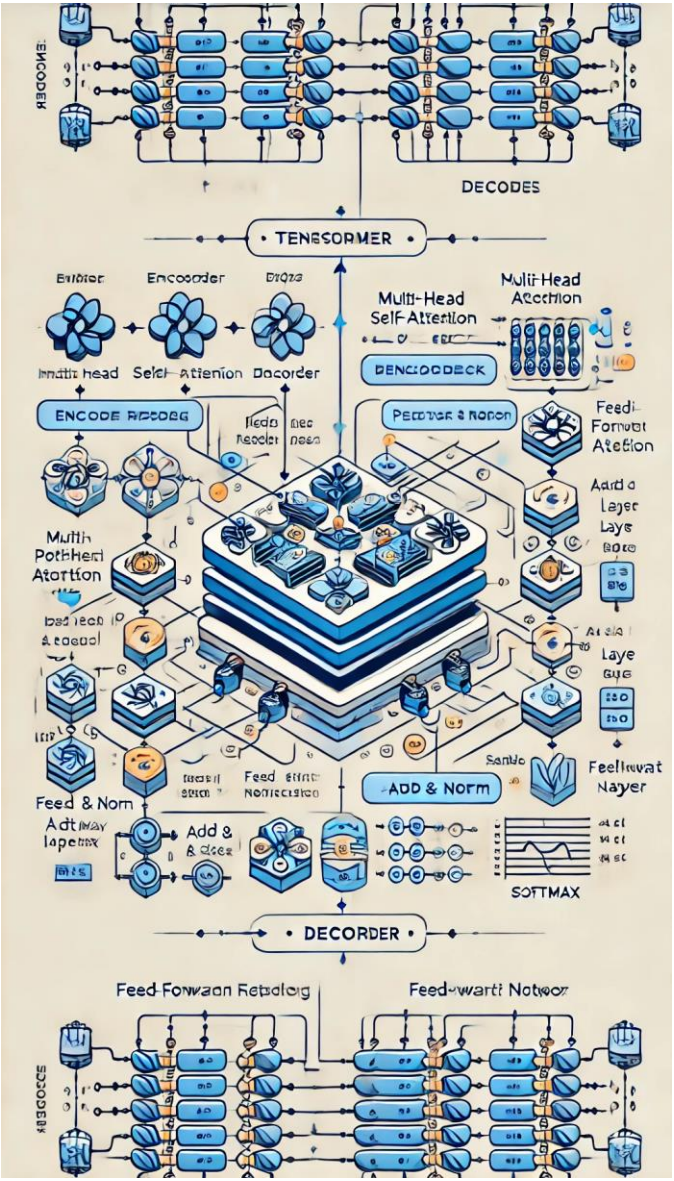
Supera i limiti delle precedenti architetture come RNN e LSTM.

Utilizza il meccanismo di attenzione, che permette di processare intere sequenze contemporaneamente anziché parola per parola.

Caratteristica principale: La capacità di catturare relazioni tra parole indipendentemente dalla loro distanza nella frase.

L'architettura del Transformer è composta da due componenti principali:
- Encoder: Processa l'input testuale e genera rappresentazioni vettoriali.
- Decoder: Utilizza le rappresentazioni dell'encoder per generare l'output (es. traduzione).

Strutture chiave:
- Multi-Head Self-Attention: Permette al modello di "prestare attenzione" a diverse parti della frase contemporaneamente.
- Feed-Forward Networks: Reti neurali dense applicate dopo il meccanismo di attenzione.
- Positional Encoding: Aggiunge informazioni sulla posizione delle parole nella sequenza.



NLP: Transformers

Il Self-Attention calcola quanto ogni parola in una frase è "rilevante" rispetto alle altre parole. Utilizza tre matrici: Q (Query)K (Key)V (Value)
La formula:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$$

d_k è una costante per la stabilizzazione numerica.

Esempio pratico: Nella frase "Il gatto mangia il pesce", il modello assegna più attenzione tra "gatto" e "mangia" rispetto a parole meno rilevanti come "il".

Modelli Basati sui Transformers

- BERT (Bidirectional Encoder Representations from Transformers): Comprende il contesto delle parole in entrambe le direzioni (sinistra e destra). Utilizzato per classificazione del testo.

- GPT (Generative Pre-trained Transformer): Utilizza un'architettura unidirezionale (causale). Ideale per generazione di testo e modelli di linguaggio.

- T5 (Text-to-Text Transfer Transformer): Modello che converte ogni task NLP in un problema "testo-come-input → testo-come-output".

- RoBERTa, XLNet, DistilBERT: Varianti ottimizzate di BERT.

A differenza degli RNN, i Transformers processano parole simultaneamente.
Velocizzano il training su grandi dataset.
Catturano relazioni complesse tra parole grazie all'attenzione multi-head.

NLP: reti neurali

Le reti neurali sono modelli ispirati al cervello umano che apprendono relazioni nei dati. Nel NLP, vengono utilizzate per elaborare e comprendere il linguaggio umano.

Obiettivo: Convertire il linguaggio naturale in una rappresentazione numerica interpretabile dai computer.

Tipi di reti principali

- Feedforward Neural Networks (FNN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Transformers

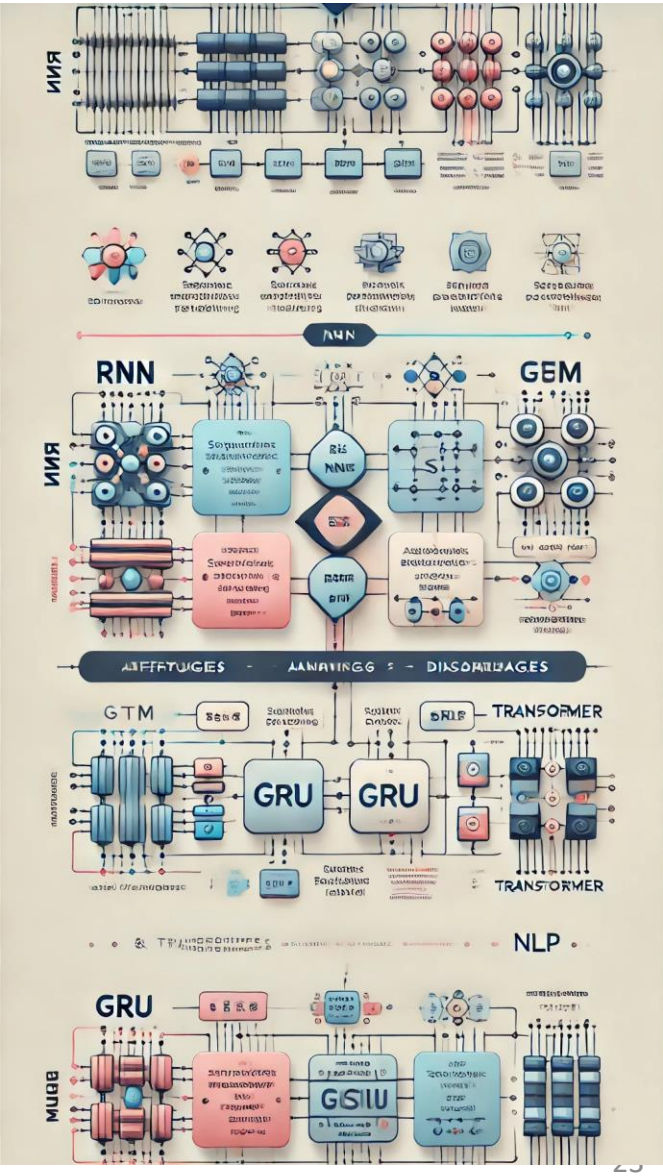
Reti Feedforward (FNN): Sono le reti neurali più semplici

Struttura: Input → Hidden Layer(s) → Output.

Limiti: Non tengono conto della sequenzialità delle parole, quindi non sono ideali per il NLP.

Esempio di applicazione

Classificazione del testo breve (BoW o TF-IDF come input).



NLP: reti neurali

Recurrent Neural Networks (RNN): Le RNN sono progettate per gestire i dati sequenziali. Ogni stato della rete dipende non solo dall'input corrente, ma anche dallo stato precedente.

Caratteristiche
- Ideali per elaborare sequenze (es. frasi, testi).
- Memoria a breve termine: Tiene traccia solo delle parole vicine.

Problema del Vanishing Gradient: La rete fatica a catturare relazioni a lungo termine.

Applicazioni
- Modelli di linguaggio (previsione della parola successiva).

Long Short-Term Memory (LSTM): Le LSTM sono un'estensione delle RNN progettate per risolvere il problema del vanishing gradient.

Caratteristiche
- Utilizzano celle di memoria per catturare relazioni a lungo termine.
- Contengono gate (Input, Forget, Output) che regolano il flusso delle informazioni.
- Riescono a gestire testi lunghi e relazioni a distanza.

Applicazioni
- Traduzione automatica.
- Analisi del sentiment.

Gated Recurrent Units (GRU)I GRU sono una versione semplificata delle LSTM. Hanno meno parametri e sono più efficienti in termini computazionali, con tempi di addestramento più rapidi.

NLP: reti neurali

Transformers risolvono i limiti delle RNN e LSTM grazie al meccanismo di attenzione:Permettono il parallelismo durante l'addestramento.

Catturano relazioni a lungo termine in modo più efficiente.

Differenza principale: Le RNN elaborano parola per parola. I Transformers elaborano tutta la sequenza contemporaneamente.

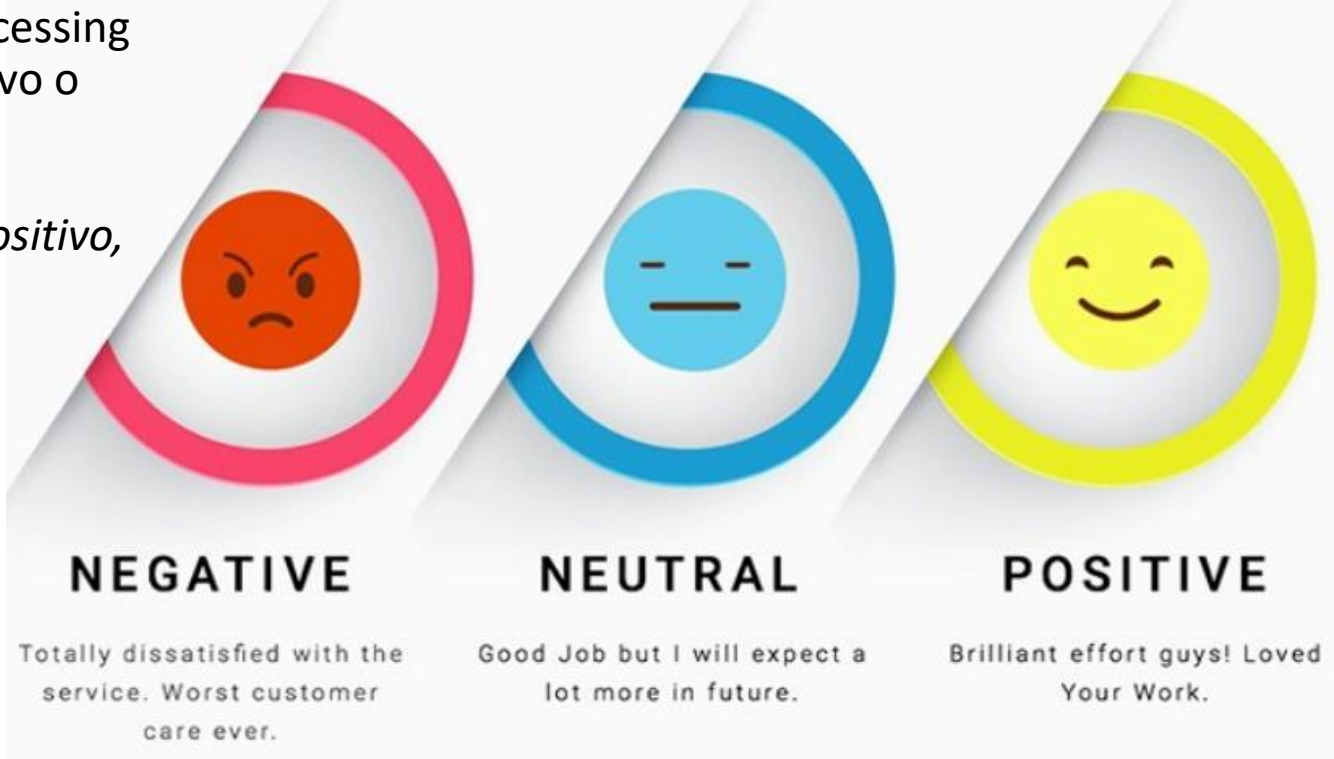
Modello	Vantaggi	Svantaggi
RNN	Gestisce sequenze brevi	Vanishing gradient, lentezza
LSTM	Cattura contesti a lungo termine	Costoso dal punto di vista computazionale
GRU	Più veloce delle LSTM	Meno potente in alcuni casi
Transformers	Parallelizzazione, relazioni globali	Richiedono grandi dataset e risorse

NLP: Sentiment Analysis

La Sentiment Analysis è una tecnica di Natural Language Processing utilizzata per identificare, estrarre e classificare il tono emotivo o l'opinione all'interno di un testo.

Obiettivo: *Determinare se un testo esprime un sentimento positivo, negativo o neutro.*

Applicazioni comuni
- Analisi delle recensioni di prodotti
- Monitoraggio delle opinioni sui social media
- Feedback dei clienti
- Analisi delle notizie



Approccio Basato su Regole: Utilizza dizionari di parole con punteggi di sentimento predefiniti. Esempio:Parole come “eccellente” e “fantastico” → positivo. Parole come “pessimo” e “orribile” → negativo.
Vantaggi: Semplice da implementare. Utile per analisi superficiali
Svantaggi: Non riconosce sarcasmo, ironia o contesto. Non gestisce espressioni complesse (es. “non male” → positivo)

NLP: Sentiment Analysis

Approccio Basato su Machine Learning: Utilizza algoritmi di apprendimento supervisionato.

Fasi principali:
- Preprocessing del testo (tokenizzazione, stemming, rimozione delle stop words).
- Estrazione delle feature: Bag of Words (BoW), TF-IDF (Term Frequency-Inverse Document Frequency), Word embeddings (Word2Vec, GloVe, FastText)
- Addestramento del modello su dati etichettati.
- Classificazione (es. sentiment positivo/negativo).

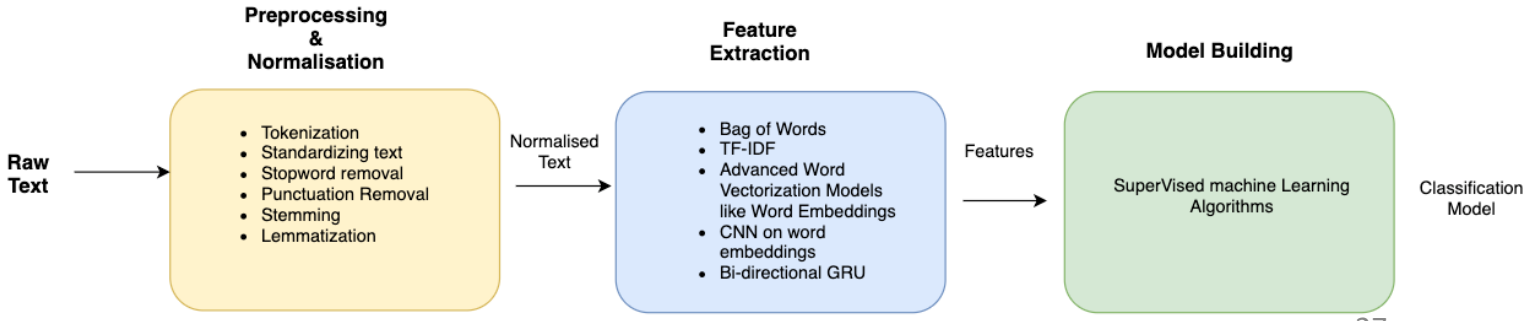
Modelli comuni
- Naive Bayes: Modello probabilistico semplice ed efficace.
- Support Vector Machines (SVM): Ideale per separare sentimenti in due categorie.
- Logistic Regression: Adatto a problemi di classificazione binaria.

Approccio Basato su Deep Learning: Utilizza reti neurali avanzate per catturare relazioni complesse nel testo.

Modelli popolari
- LSTM e GRU: Catturano dipendenze a lungo termine nel testo.
- CNN (Convolutional Neural Networks): Utilizzano convoluzioni per estrarre pattern locali.
- Transformers: Modelli come BERT, RoBERTa e GPT forniscono risultati all'avanguardia.

Vantaggi
- Gestisce testi complessi, sarcasmo e contesto.
- Cattura la semantica profonda.

Svantaggi
- Richiede grandi quantità di dati per l'addestramento.
- Elevati costi computazionali.

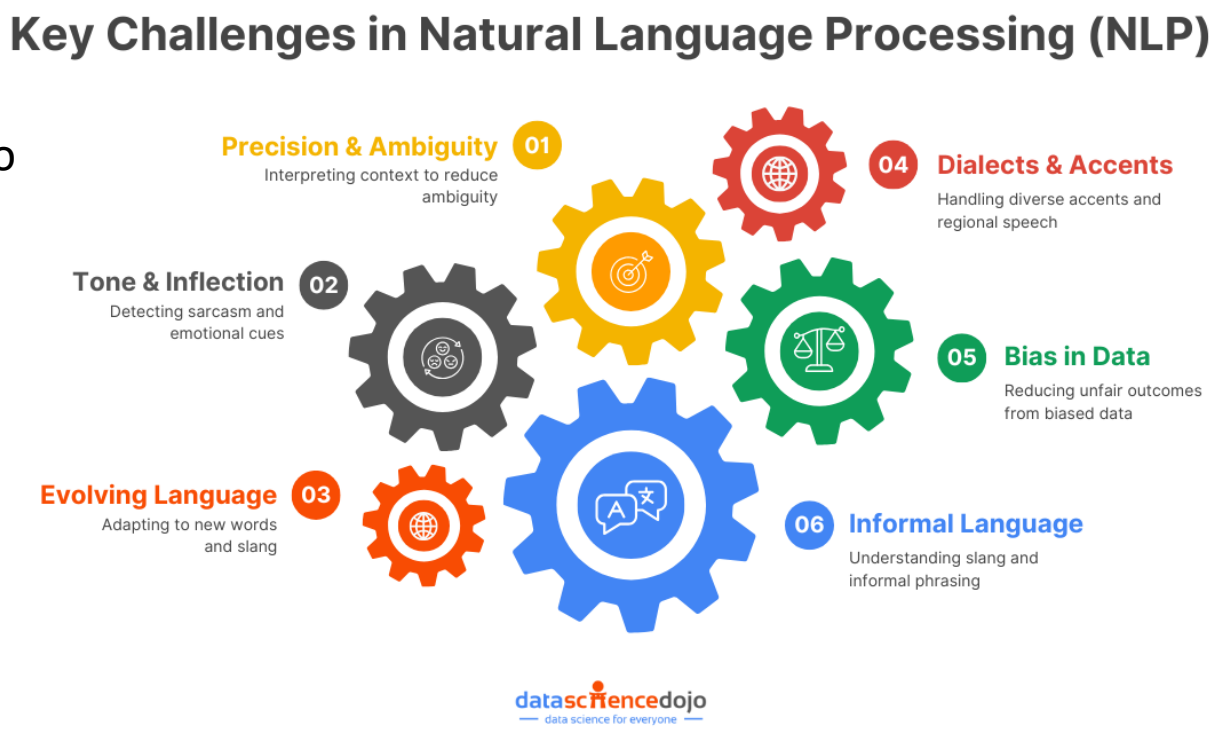


An NLP Model Creation Pipeline

NLP: Sentiment Analysis

Sfide della Sentiment Analysis

- Ambiguità: Frasi ambigue o contesti multipli possono confondere i modelli: “Il film era una bomba” → Positivo o negativo?
- Sarcasmo e Ironia: Interpretare frasi sarcastiche è difficile (es. “Che bella giornata di pioggia!”).
- Parole fuori vocabolario (OOV): Termini nuovi o slang non riconosciuti possono compromettere l’accuratezza.
- Contesto e negazioni: Frasi come “Non è affatto male” richiedono analisi contestuali avanzate.



La Sentiment Analysis è uno strumento cruciale nel NLP, con applicazioni concrete in molti settori. L'evoluzione verso modelli basati su Deep Learning ha reso questa tecnica più potente, ma restano sfide come il sarcasmo e l'ambiguità.

NLP: Etica e Bias

Etica e Bias nel NLP

L'etica nel NLP riguarda le implicazioni morali e sociali derivanti dall'utilizzo delle tecnologie di elaborazione del linguaggio naturale. Alcune questioni chiave includono:

- Bias nei dati e nei modelli.
- Privacy degli utenti.
- Uso improprio delle tecnologie NLP (es. deepfakes, manipolazione delle informazioni).
- Accessibilità e discriminazione.

Il bias si verifica quando i modelli NLP riflettono e amplificano pregiudizi presenti nei dati su cui sono stati addestrati. Questi pregiudizi possono essere di tipo:

- Sociale: discriminazione basata su genere, etnia, religione, orientamento sessuale, ecc.
- Culturale: privilegiare lingue o dialetti dominanti a scapito delle minoranze.
- Linguistico: difficoltà nell'elaborare slang, linguaggi informali, o lingue a bassa disponibilità di dati.

NLP: Etica e Bias

Cause del Bias nei Modelli
I dati spesso provengono da fonti reali (es. social media, articoli) che riflettono pregiudizi esistenti.

Esempio: Generazione di frasi stereotipate ("Lui è un ingegnere, lei è un'insegnante").

Algoritmi e Modelli: Alcuni modelli amplificano i bias presenti nei dati a causa della loro architettura e obiettivi di ottimizzazione. Modelli di Word Embedding come Word2Vec e GloVe, catturano relazioni pregiudizievoli tra parole.
Esempio: Vettori che associano "uomo" a "carriera" e "donna" a "famiglia".

Annotazione dei Dati: I pregiudizi possono derivare anche dai bias cognitivi degli annotatori umani.

Implicazioni Etiche del Bias
Discriminazione: Sistemi NLP discriminatori possono penalizzare gruppi minoritari.
Esempio: Sistemi di selezione automatica del personale che escludono candidati sulla base del genere o etnia.

Manipolazione: NLP può essere usato per generare disinformazione o contenuti fuorvianti (es. fake news).

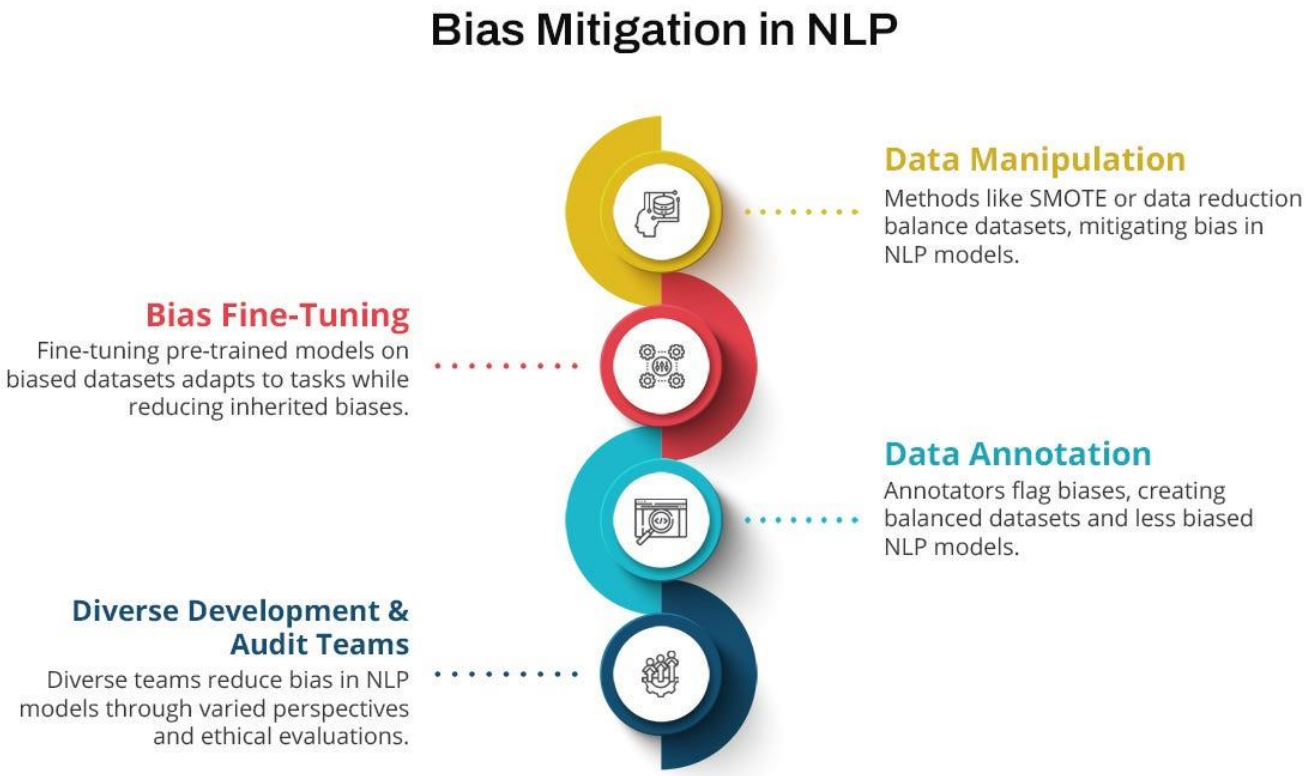
Esclusione: Lingue non ampiamente rappresentate nei dati rischiano di essere ignorate nei modelli avanzati.

Violazione della Privacy: Analisi dei testi può rivelare informazioni personali sensibili.

NLP: Riduzione del Bias

Soluzioni per Ridurre il Bias nel NLP

- **Raccolta di dati bilanciati**: Assicurare la rappresentazione equa di tutti i gruppi (genere, etnia, cultura).
- **Rimozione di dati problematici**: Identificare e rimuovere dati che contengono pregiudizi espliciti.
- **Formare gli annotatori** per ridurre bias inconsci.
- **Tecniche di Debiasing** nei Modelli Post-Processing dei Word-Embeddings per neutralizzare pregiudizi.
- **Fairness Regularization**: Implementare vincoli etici nell'addestramento dei modelli.
- **Retraining** con dati più equi e rappresentativi.
- **Valutazione della Fairness**: Creare metriche che misurino bias nei modelli (es. differenze di accuratezza tra gruppi).
- **Promuovere politiche** di open-source e audit indipendent
- **Bias dinamico**: I pregiudizi evolvono nel tempo (linguaggic mutevole).
- **Modelli di Grandi Dimensioni**: Modelli come GPT-3 e GPT 4 sono difficili da monitorare completamente.
- **Equità Multilingue**: Ridurre il bias in lingue con dati limitati.



NLP: Debiasing – codice python

```
from gensim.models import Word2Vec
import numpy as np

# Caricare un modello Word2Vec pre-addestrato
model = Word2Vec.load("word2vec.model")

# Parole associate a bias di genere
male_words = ["he", "man", "male"]
female_words = ["she", "woman", "female"]

# Calcolare vettore di genere
gender_axis = np.mean([model[w] for w in male_words], axis=0) - \
          np.mean([model[w] for w in female_words], axis=0)

# Funzione per neutralizzare il bias
def debias_word(word, gender_axis):
   word_vector = model[word]
   projection = np.dot(word_vector, gender_axis) / np.linalg.norm(gender_axis)
   return word_vector - projection * gender_axis

# Neutralizzare bias per una parola
neutralized_vector = debias_word("nurse", gender_axis)
print("Bias neutralizzato per 'nurse'")
```

Esempio Pratico: Debiasing nei Word Embeddings: Implementazione in Python per mitigare bias nei vettori di parole (usando Gensim)

NLP: Conclusione

Nel corso di questa presentazione, abbiamo esplorato:

- **Fondamenti del NLP**
  - Definizione, obiettivi e applicazioni

- **Tecniche principali**
  - Tokenizzazione
  - stemming
  - lemmatizzazione
  - word embeddings
  - sentiment analysis

- **Modelli avanzati**
  - RNN
  - LSTM
  - GRU
  - Transformer (BERT, GPT).

- **Sfide etiche**
  - Bias
  - equità e trasparenza nei modelli NLP

Benefici del NLP

- **Automazione**: Riduzione dei tempi per attività linguistiche complesse (Automatizzare risposte nei chatbot)

- **Accessibilità**: Traduzione automatica, trascrizione in tempo reale, interfacce vocali migliorano l'accesso alle informazioni.

- **Personalizzazione**: Sistemi di raccomandazione e assistenti virtuali su misura.

- **Analisi dei Dati Testuali**: Estrarre informazioni preziose da grandi volumi di dati non strutturati.

Stiamo costruendo sistemi che non solo capiscono parole, ma che comprendono significati, intenzioni ed emozioni. Il futuro del NLP dipende dall'equilibrio tra innovazione tecnologica e responsabilità etica.

NLP: sfide presenti e future

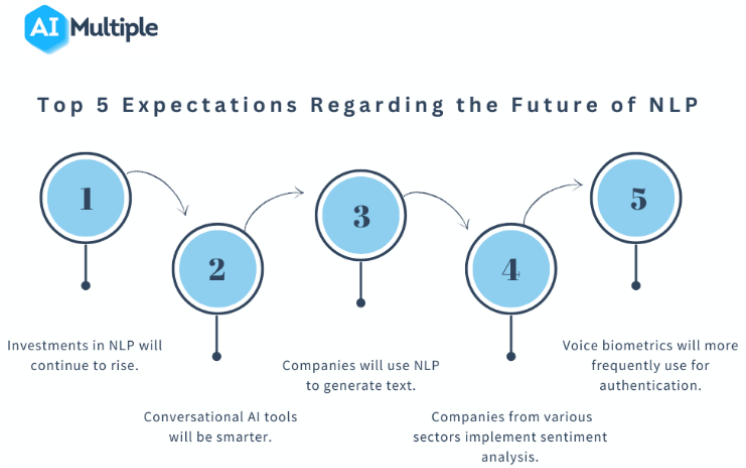
- Ambiguità del linguaggio: Interpretazione del contesto, sarcasmo e linguaggio figurativo.

- Bias nei dati e modelli: Modelli NLP amplificano pregiudizi esistenti.

- Lingue con risorse limitate: Mancanza di dati per lingue minoritarie o dialetti.

- Costi computazionali: Addestrare modelli di grandi dimensioni (es. GPT-4) richiede risorse significative.

- Privacy e sicurezza: Rischio di uso improprio (es. phishing generato automaticamente).

Prospettive Future del NLP

- Miglioramento dei Modelli con Modelli Multimodali: Integrazione di linguaggio, immagini, video e segnali audio (es. CLIP, GPT-4 Vision).
- Modelli Multilingue: Sistemi NLP che comprendono e generano testi in più lingue senza bias culturali.
- Modelli più efficienti: Riduzione del consumo energetico e dei costi computazionali tramite modelli leggeri come DistilBERT.
- Approfondimento della Comprensione Contestuale: Creare modelli in grado di comprendere l'intento e il significato profondo delle frasi (es. sarcasmo, emozioni).
- Progressi nell'Explainable AI (XAI) per migliorare la trasparenza delle decisioni dei modelli.
- Equità e Inclusione
- Debiasing avanzato: Tecniche per rimuovere pregiudizi sistemici dai modelli e dai dati.
- Supporto alle lingue poco rappresentate: Promozione della diversità linguistica e culturale.